

开题报告

深度学习核心原理预研与实践

随着人工智能技术的快速发展，深度学习已成为解决复杂问题的核心工具，尤其在计算机视觉、自然语言处理等领域展现出强大的能力。我们致力于手写一个简单的神经网络，为进一步理解并应用DL打下基础。

本报告将从以下三个维度展开：

- 核心原理预研
- 技术现状：激活函数与优化器总结
- 分工与实验规划

一、核心原理预研

1. 神经网络基本概念

- 输入层、隐藏层、输出层**：神经网络由多层神经元组成，输入层接收特征，隐藏层自动学习高层特征，输出层给出预测。
- 黑盒特性**：隐藏层学到的特征可能难以解释，但通常对预测非常有用。

2. 激活函数与前向传播

单个神经元公式：

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}, \quad a^{[l]} = g(z^{[l]})$$

- $W^{[l]}$: 权重矩阵 $n^{[l]} \times n^{[l-1]}$
- $b^{[l]}$: 偏置向量 $n^{[l]} \times 1$
- $g(z)$: 非线性激活函数
 - Sigmoid**: $g(z) = 1/(1 + e^{-z})$
 - ReLU**: $g(z) = \max(0, z)$
 - tanh**: $g(z) = \tanh(z)$

作用：激活函数引入非线性，使网络能够拟合复杂函数.没有非线性激活，多层网络退化为线性模型，失去表达能力。

3. 向量化

矩阵化实现：

$$Z^{[l]} = W^{[l]}A^{[l]} + b^{[l]}, \quad A^{[l+1]} = g(Z^{[l]})$$

作用：矩阵化实现提高计算效率，尤其在 NumPy 中对大规模数据集训练至关重要。

4. 损失函数

MNIST数据集是一个多项分类问题,应用**Softmax Regression**

$$\hat{y}_{i,k} = \frac{e^{z_{i,k}}}{\sum_{j=1}^C e^{z_{i,j}}}$$
$$L = -\frac{1}{N} \sum_{i=1}^N \log \hat{y}_{i,y_i}$$

5. 反向传播与链式法则

5.1 链式法则公式

$$\frac{\partial L}{\partial W^{[l]}} = \frac{\partial L}{\partial a^{[L]}} \cdot \frac{\partial a^{[L]}}{\partial z^{[L]}} \cdots \frac{\partial z^{[l]}}{\partial W^{[l]}}$$

5.2 单隐藏层示例

- 输出层梯度：

$$dZ^{[2]} = A^{[2]} - Y \quad (1 \times m)$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} (A^{[1]})^T \quad (n_y \times n^{[1]})$$

$$db^{[2]} = \frac{1}{m} \sum_{i=1}^m dZ^{[2]} \quad (n_y \times 1)$$

- 隐藏层梯度：

$$dZ^{[1]} = (W^{[2]})^T dZ^{[2]} \circ g'(Z^{[1]}) \quad (n^{[1]} \times m)$$

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T \quad (n^{[1]} \times n_x)$$

$$db^{[1]} = \frac{1}{m} \sum_{i=1}^m dZ^{[1]} \quad (n^{[1]} \times 1)$$

6. 参数更新与优化器

核心原理是利用梯度下降求解最值问题

- **SGD(Stochastic Gradient Descent)**：

$$W^{[l]} := W^{[l]} - \alpha dW^{[l]}, \quad b^{[l]} := b^{[l]} - \alpha db^{[l]}$$

- **Mini-batch Gradient Descent**：每次更新使用 B 个样本，提高效率
- **Momentum**：

$$v_{dW^{[l]}} = \beta v_{dW^{[l]}} + (1 - \beta) dW^{[l]}, \quad W^{[l]} := W^{[l]} - \alpha v_{dW^{[l]}}$$

7. 参数初始化

- 避免对称性问题：随机小值初始化 $N(0, 0.1)$
- **Xavier/He 初始化**：

$$W^{[l]} \sim N\left(0, \frac{2}{n^{[l]} + n^{[l-1]}}\right)$$

8. 正则化

- **L2正则化**：

$$J_{L2} = J + \frac{\lambda}{2} \sum_l \|W^{[l]}\|^2$$

- 更新公式：

$$W^{[l]} := (1 - \alpha \lambda)W^{[l]} - \alpha dW^{[l]}$$

作用：限制权重过大，防止过拟合。

二、技术现状：常见激活函数

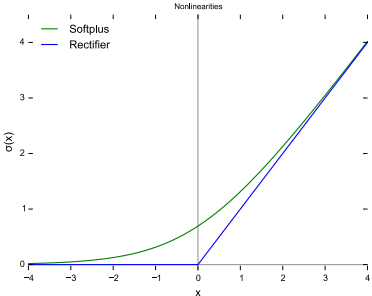
激活函数作用

激活函数（Activation Function）用于为神经网络引入非线性能力。若没有激活函数，无论网络多深，本质仍是线性变换：

$$f(x) = Wx + b$$

无法拟合复杂非线性数据。

1. ReLU

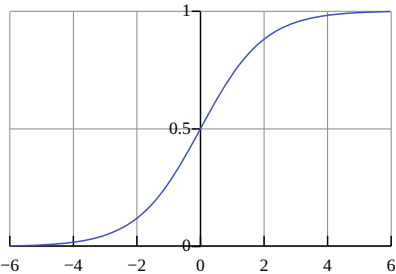
项目	内容
数学公式	$f(x) = \max(0, x)$
函数图像	

ReLU 是现代神经网络中最常用的隐藏层激活函数之一。它的优点是计算简单，前向传播和反向传播都很高效。当 $x > 0$ 时，ReLU 的导数为 1，因此相比 Sigmoid 和 Tanh，它可以在一定程度上缓解梯度消失问题。

ReLU 的另一个优点是会产生稀疏激活。对于小于等于 0 的输入，输出为 0，这使得一部分神经元在某些样本上不被激活。

但是，ReLU 也有缺点。最典型的问题是“dying ReLU”。如果某个神经元的输入长期小于等于 0，那么它的输出一直为 0，梯度也一直为 0，这个神经元可能再也无法被更新。

2. Sigmoid

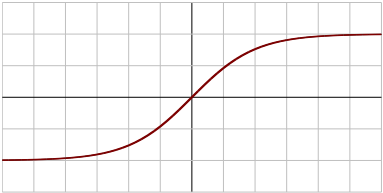
项目	内容
数学公式	$f(x) = \frac{1}{1+e^{-x}}$
函数图像	

Sigmoid 的优点是输出可以被解释为概率，因此早期常用于二分类任务的输出层。它函数平滑、可导，数学性质较好。

但是，Sigmoid 的主要缺点是容易产生梯度消失。当输入 x 很大或很小时，Sigmoid 的输出会接近 1 或 0，此时导数会非常接近 0，导致反向传播时梯度变得很小，前面层的参数更新缓慢。另外，Sigmoid 的输出不是以 0 为中心，这可能会影响优化效率。

因此，Sigmoid 一般不适合作为深层神经网络隐藏层的默认激活函数，但在本次实验中也会被引入作为变量之一。

3. Tanh

项目	内容
数学公式	$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
函数图像	

Tanh 相比 Sigmoid 的一个优点是输出以 0 为中心，因此通常比 Sigmoid 更有利于优化。

但是，Tanh 仍然存在梯度消失问题。当输入的绝对值很大时，Tanh 的输出会接近 -1 或 1，导数接近 0。因此，在较深的网络中，Tanh 也可能导致前面层学习速度变慢。

三、技术现状：常用优化器

优化器（Optimizer）是深度学习中用于更新模型参数、最小化损失函数的核心算法。不同优化器的参数更新方式不同，因此在收敛速度、训练稳定性、泛化能力和适用场景上存在明显差异。

1. 基础梯度下降类

这类优化器的核心区别在于：**每次计算梯度时所使用的样本数量不同**，如使用全部训练数据计算梯度的 Batch Gradient Descent（BGD），每次仅使用1个样本更新参数的 Stochastic Gradient Descent（SGD），以及混合的Mini-batch SGD（MB-SGD）。

一般合理对数据切片后采用Mini-batch SGD（MB-SGD）。

2、带动量的梯度下降类

核心思想

为了解决SGD震荡严重、收敛缓慢的问题，引入“动量（Momentum）”机制，使参数更新具有惯性。

$$v_t = \mu v_{t-1} - \eta \nabla_{\theta} L(\theta_{t-1})$$
$$\theta_t = \theta_{t-1} + v_t$$

4 ·Adam系列优化器

核心思想

在“动量优化”进行“自适应学习率”。

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

四、分工细节

成员	分工细节
武钰昕	完成最初项目结构搭建；完成 nn/ 核心模块开发及相关测试；完成后续加分项内容开发
梁新越	负责 MNIST 数据读取、预处理、训练/验证/测试集划分、mini-batch DataLoader；后续承担 Fashion-MNIST 扩展实验的数据接入；配合完成不同 batch size，learning rat的实验对比。
许海东	完成不同隐藏层结构和optimizer的实验对比；承担文档整理工作，撰写开题报告，补充公式推导，整理训练曲线、PPT 和演示材料。

协作方式

本项目使用 **Git + GitHub** 进行代码管理和小组协作。

五、任务进度表

本项目计划按 5 周推进，覆盖“基础 MLP 准确率达标”和两个加分项。

周次	阶段目标	主要任务	阶段交付物
第 1 周	环境准备与核心原理验证	完成项目目录设计；明确数据格式约定；实现并测试 Linear、ReLU、SoftmaxCrossEntropyLoss、SGD、Sequential；使用随机假数据跑通 forward -> loss -> backward -> update。	项目骨架；nn/ 核心模块；单元测试；随机数据前后向传播验证结果。
第 2 周	数据模块与基础训练跑通	完成 MNIST 数据读取、展平、归一化和 train/validation/test 划分；实现 DataLoader 的 shuffle 和 mini-batch；编写 train.py 和 evaluate.py 的基础流程；使用小模型 784 -> 128 -> 10 验证 loss 能下降、accuracy 能上升。	data/ 模块；基础训练脚本；验证集 accuracy 记录。
第 3 周	准确率达标与超参数实验	使用主模型 784 -> 256 -> 128 -> 10 训练 MNIST；调试学习率、batch size、初始化方法和训练轮数；绘制 Loss 与 Accuracy 曲线；测试集 accuracy 达到 95% 以上。	达标模型；训练曲线；学习率、batch size、网络深度对比实验；阶段性实验分析。
第 4 周	加分项一：BatchNorm	在 NumPy 框架中实现 BatchNorm 层，完成 forward/backward；加入 Linear -> BatchNorm -> ReLU 的网络结构；比较有无 BatchNorm 时的收敛速度、验证集准确率和训练稳定性。	BatchNorm 层；相关测试；有/无 BatchNorm 的实验曲线和分析。
第 5 周	加分项二与结项交付	接入 Fashion-MNIST 数据集，复用自研 MLP 框架完成训练；比较 MNIST 与 Fashion-MNIST 的准确率差异和原因；完善实验报告、演示视频和答辩 PPT；整理最终代码和压缩包。	Fashion-MNIST 实验结果；最终实验报告；演示视频素材；答辩 PPT；最终提交压缩包。